

ELCT 302
Laboratory Report #3

Control Design and Implementation

Group #2
3/14/23

I hereby certify that I have complied
with the Spirit and the Letter
of the Carolinian Creed
in preparing this report

Phillip Craft

Sydney Powers

Chris Chau

Department of Electrical Engineering
University of South Carolina
Columbia, SC 29208

Abstract

This lab report will be discussing the processes that steering group #2 went through to complete the assigned objectives of this of this project. The introduction will explain the tasks completed in the lab; the theory section will discuss the concepts that the tasks are based off of. This answers the question of why things are done the way they are. In the procedure, we discuss the tasks that were accomplished and the methods that were used in the process. This will be followed by a results section that will show the measurements and data that we gathered as well as the figures that we use to demonstrate our work.

Table of Contents

Table of Figures.....	3
I Introduction	4
II Theory	4
A. PD Control System.....	4
B. PD Control Design Equations	5
C. Control Design Parameters.....	5
D. Continuous Time Stability Analysis	6
E. Step Response Characterization	7
F. Determination of Controller Stability in Discrete Time.....	8
G. Derivative Approximation Method	8
H. Steady State Error.....	9
I. Derivative Kick	10
III Experimental or Procedure.....	10
A. Steering Control Design	10
B. Steering Control Simulation	10
C. Control Implementation	11
IV Discussion / Results.....	11
A. MATLAB Output	11
B. Simulink Output.....	14
V Conclusions.....	15
Conclusion	15
VI Appendix.....	16
A. MATLAB Code	16
B. Mbed Code.....	17
C. PD Control Design Derivation	17
VII References.....	19

Table of Figures

Figure 1 – Block Diagram of PD Controller Used for Derivation Equations	5
Figure 2 – Graph Depicting the Gain and Phase Crossover, Gain and Phase Margin	6
Figure 3 – Example of Step Response with Shown Characteristics	8
Figure 4 – Example of Unit Step Input with Steady State Error of 0	9
Figure 5 – Bode Plot of the Open-Loop System	12
Figure 6 – Nyquist Plot of the Open-Loop System	13
Figure 7 – Step Response of Closed-Loop System	13
Figure 8 – Simulink Block Diagram	14
Figure 9 – Simulink Output Without Disturbance	14
Figure 10 – Simulink Output with Disturbance	15

Laboratory Three

I Introduction

In this report, we will walk through three major parts of the lab project. We will start by talking about the theory. This section will cover all of the reasoning and explanation behind what we do in the lab. We will cover, PD Control Systems, PD Control Design Equations, Control Design Parameters, Continuous Time Stability Analysis, Step Response Characterization, Determination of Controller Stability in Discrete Time, Derivative Approximation Method, Steady State Error, and Derivative Kick. Each of these will be discussed in depth along with explanation of how they pertain to the lab.

The next major part of the lab is the procedure. This section has three subsections: Steering Control Design, Steering Control Simulation, and Control Implementation. In the design part, we will discuss the constant values that had to be calculated as well as the starting parameters that were chosen. The simulation part will talk about what we did with Simulink. The implementation part will go through what was done with the coding of the microcontroller.

The final major section of the report is the discussion section. In this we will walk through the results of the simulations and implementation done in the lab. This part mostly contains plots and figures along with explanation of what each of them represents.

II Theory

A. PD Control System

In a proportional controller, the output of the controller is proportional to the input [2]. In a derivative controller, the output is proportional to the derivative of the input, or the rate of change [2]. A PD, or Proportional Derivative controller is the combination of both. It is a controller in a system that is based on the proportion to the error signal and the derivative of the error signal [2].

This PD controller is more efficient than either the proportional or derivative controller alone [2]. The derivative portion of the controller is aimed to change the output with the error [2]. The issue is that when the error is steady, unchanging, then the derivative is zero [2]. The proportional aspect in addition to the derivative part will help the PD controller also consider constant error [2]. The general equation for a PD controller is given below.

$$u(t) = k_p e(t) + k_D \dot{e}(t) \quad \text{Equation 1}$$

The first term in Equation 1 is the proportional term. The purpose of this is to reduce the rise time and steady state error [3]. Unfortunately, this also increases the overshoot of the signal [3]. The derivative aspect of the controller is the second term in Equation 1. This term is there to increase the stability of the system and improve the transient response by reducing the overshoot caused by the proportional controller [3].

B. PD Control Design Equations

For a PD controller, two design equations involving two constants, K_d and K_p can be derived by looking at the equations in Section C of the Appendix. First of all, “ K_p is the constant of proportionality of error signal [...] K_d is the constant of proportionality of derivative of the error signal” [13]. Looking at the equation on Line 1, it is the forward path gain before adding a controller, which a block diagram of a system that can be seen on Figure 1. Line 2 is the forward path gain with the controller which is just Line 1 multiplied by the controller’s gain, denoted by G_c . Line 3 is the forward path gain at the crossover frequency, ω_c , in exponential form, providing the magnitude and phase of the gain. Line 4 is definition of the forward path gain with the controller at the crossover frequency. Line 5 is using the forward path gain from Line 2 and making it equal to the definition of the forward path gain from Line 4. The forward path gain is also in exponential notation, Line 3 is used for both the controller gain and the forward path gain.

Line 6 is isolating the controller’s gain, so the forward path gain before the controller in exponential form is divided on both sides. Simplifying with exponential rules, the phase are subtracted, so the forward path phase can be moved to the numerator and be subtracted by the phase of the definition of the crossover, and the magnitude of the forward path gain stays in the denominator. Line 7 is the definition of Euler’s Formula, which is used in Line 8 on the right side of the equation. The exponential form is in Euler’s form. On the left side of Line 8, it is the definition of a PD controller gain, where K_p is the proportional gain constant and K_d is the derivative gain constant. By looking at Line 8, the definition of the of the PD controller gain looks similar to the Euler’s form of the forward path gain (right side of Line 8), so Line 9, is comparing both sides of Line 8 for K_p , and that is the design equation for K_p . Line 10 is doing the same for K_d , and then Line 11 is dividing the $j\omega_c$ by both sides to get the K_d design equation.

From the PD control design equations for K_p and K_d , Lines 9 and Lines 11, respectively, they are dependent of the magnitude and phase of the crossover frequency and the phase margin. Choosing the crossover frequency and phase margin accordingly for the system allows for the proportional and derivative gain constants to be best effective for the system.

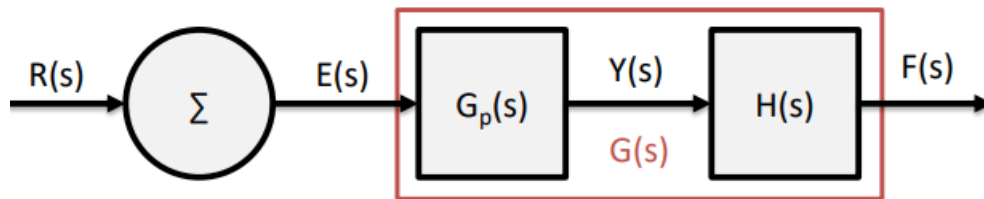


Figure 1: Block Diagram of PD Controller Used for Derivation Equations

C. Control Design Parameters

Control design parameters include crossover frequencies and margins. For both of these parameters, it applies to gain and phase. The crossover frequency is different for the gain and the

phase, the gain crossover frequency is the frequency where the gain crosses the 0 dB line or the gain of 1 (unity gain) and the phase crossover frequency is the frequency where the phase reaches -180 degrees[11]. As for the gain margin and the phase margin, the gain margin “indicates how much gain can be added to the system before it becomes unstable [...] the phase margin tells us how much additional phase shift can occur before the 0 dB, -180 degree condition is reached and the system becomes unstable” [11]. The gain margin is determined by looking at the phase crossover frequency. At this frequency, the gain margin from the gain curve is the distance from the gain to the 0 dB line. Likewise, the phase margin can be decided by looking at the gain cross over frequency. Looking at the phase curve, the phase margin is the distance from the curve to the -180 degree line. Figure 2 shows all of the parameters. It is important to choose the crossover frequency and phase margin accordingly because the crossover frequency allows the system to have the fastest settling time when a step input occurs and a lower phase margin allows for longer ringing and a higher phase margin causes a longer time to reach the step input’s max voltage.

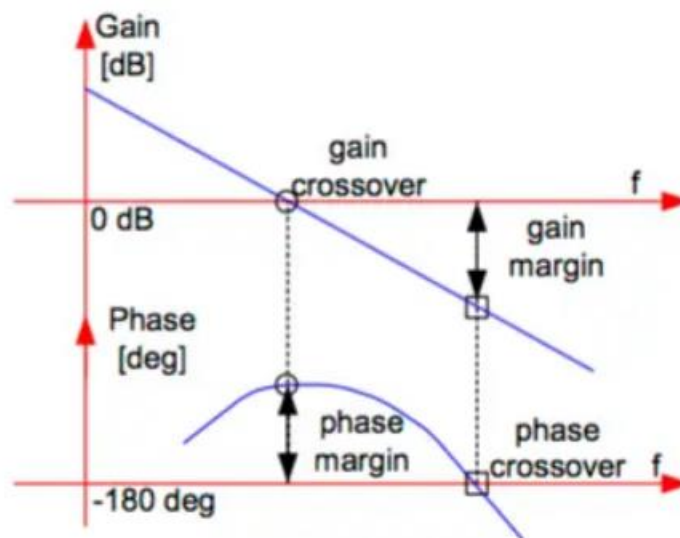


Figure 2: Graph Depicting the Gain and Phase Crossover, Gain and Phase Margin [11]

D. Continuous Time Stability Analysis

A Bode plot can be used to analyze the stability of a system. The two factors in the plots that contribute to this value are the gain margin (GM) and the phase margin (PM) [6]. The gain margin is the amount of gain in the system that can be increased or decreased without making the system unstable [6]. Consequently, a greater phase margin results in greater stability. This value can be taken from the Bode plot by calculating the vertical (y) distance between the magnitude curve on the magnitude plot, and the magnitude value at the x-point on the phase plot when the frequency is equal to 180° [6]. This can also be found by calculating the negative value of gain (in dB).

Similar to the GM, an increase in phase margin (PM) results in an increase in stability [7]. This value, expressed in degrees, is the amount of phase that can be increased or decreased before the system becomes unstable [7]. This value can be taken from the Bode plot by calculating the vertical (y) distance between the phase curve on the phase plot, and the phase value at the x-point

on the magnitude plot when the magnitude is equal to 0dB [7]. The value of phase margin is also equal to the phase lag plus 180° [7].

A closed-loop control system can also be tested for stability by examining the open-loop system's Nyquist plot. The criterion works on the argument that if there are P-number of poles on the right side of the s-plane in an open-loop system, and Z-number of zeros on the right half of the s-plane in the closed-loop system, then the number of clockwise encirclements made by the Nyquist plot is equal to the difference between 'Z' and 'P' [8]. This formula is shown below.

$$N = Z - P \quad \text{Equation 2}$$

The stability of the system can be determined through the values of gain crossover frequency, phase crossover frequency, gain margin, and phase margin [8]. The phase crossover frequency is the frequency at which the Nyquist plot crosses the real axis [8]. The gain crossover frequency is the frequency at which the magnitude of the Nyquist plot is equal to one [8]. If the phase crossover frequency is greater than the gain crossover frequency, then the system is stable [8]. If the phase crossover frequency is less than the gain crossover frequency, then the system is unstable [8]. The system is marginally stable if the frequencies are equal [8].

Another way to determine stability using a Nyquist plot is through the values of phase and gain margin. The gain margin is the reciprocal of the magnitude of the plot at the phase crossover frequency, and phase margin is the sum of 180° plus the phase angle at the gain crossover frequency [8]. If the gain margin is greater than one, and the phase margin is positive, then the system is stable [8]. If the gain margin is equal to one, and the phase margin is equal to zero, then the system is marginally stable [8]. If the gain margin is less than one, and/or the phase margin is negative, then the system is unstable [8].

E. Step Response Characterization

A step response is the output of a system where an input of a signal goes from zero to one instantaneously in the time domain. With a step response, there are five characteristics that go along with it, including, the rise time, the setting time, the overshoot, the undershoot, and the steady-state error. Written in the Control Systems Engineering textbook, "Rise time is defined as the time for the waveform to go from 0.1 to 0.9 of its final value" [6]. The final value would be the value the step response goes to when the time goes to infinity. So, the rise time is the time interval from ten percent to ninety percent. Next is the setting time, which is the time from the beginning to the time when the response reaches its final value and stays constant at that final value for two percent [6]. Overshoot is when the response goes above the final value. There can be multiple overshoots in a step response, and the largest overshoot is known as the maximum overshoot. The undershoot is the same as the overshoot, but it is when the response goes below the final value. Finally, the steady-state error is the difference between the input and the output of the system and the limit is taken with the time going to infinity. In simple terms, it's the difference

between the wanted value and the actual value of the response when it has reached steady state. Figure 3 shows the rise time, setting time, overshoot, and undershoot.

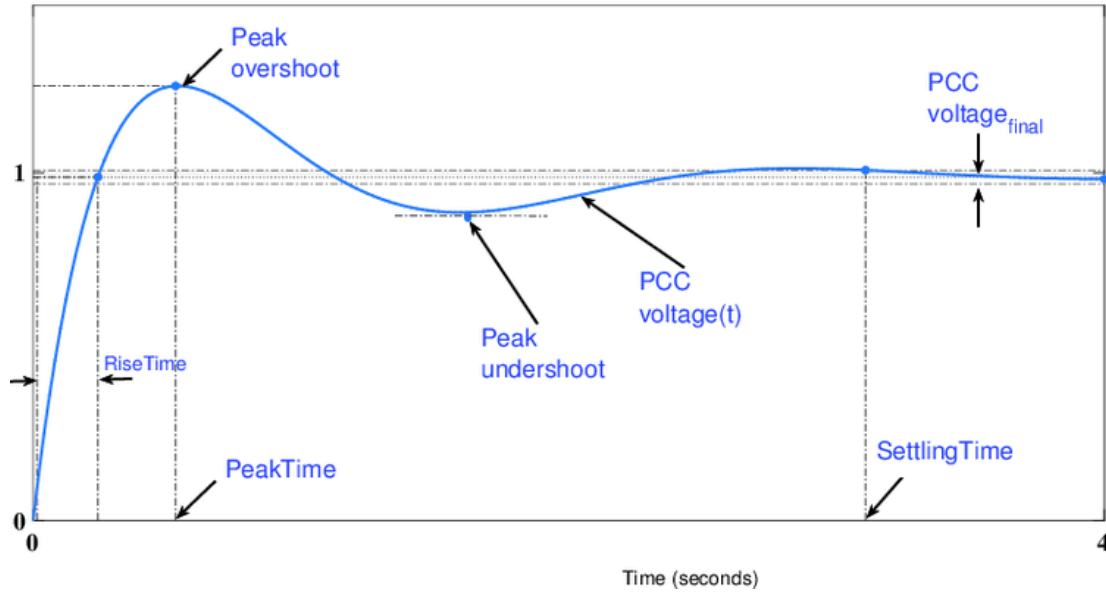


Figure 3: Example of Step Response with Shown Characteristics [6]

F. Determination of Controller Stability in Discrete Time

Determining stability within a discrete time domain can be very complicated. There are long articles explaining it with various methods that can be used. One example is the Lyapunov method. “There is a discrete version of the Lyapunov stability theorem that applies to digital systems. Given the discrete Lyapunov equation:

$$A^T M A - M = -N \quad \text{Equation 3}$$

We can use this version of the Lyapunov equation to define a condition for stability in discrete-time systems: A digital system with the system matrix A is asymptotically stable if and only if there exists a unique matrix M that satisfies the Lyapunov Equation for every positive definite matrix N”[14].

For our implementation, we are using a microcontroller with code that is inherently written in the discrete domain. Rather than using the complicated methods for calculating stability, we use simulations and testing to determine stability. One of these methods is to use Simulink to model our system and simulated it running. Simulink is in the discrete domain, so when we use it to show the transient response for example of our system, we can see whether it is stable. Another way we test the stability of the steering system is to actually place the car on track and see if it runs in a stable way when it is pushed. There will be more details given for both of these methods of testing later in the procedure and results sections.

G. Derivative Approximation Method

“A Taylor series is a series expansion of a function about a point. A one-dimensional Taylor series is an expansion of a real function “ $f(x)$ ” about a point $x = a$ is given by the equation in Figure 1” [9]. The purpose of the derivative approximation in our project is to control the error

in our system by taking the derivative of the error over time. If the error decreases with respect to time, the system will not need to make large corrections. However, if the error increases with respect to time, the system input/corrections will need to be much larger. The Taylor series is useful for this method because it helps to expand the data around one specific point.

$$f(x) = f(a) + f'(a)(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \frac{f^{(3)}(a)}{3!}(x-a)^3 + \dots + \frac{f^{(n)}(a)}{n!}(x-a)^n + \dots$$

Equation 4

This method of keeping track of error is more effective than manually correcting the system every time the car gets slightly off course. In that situation, the car would be constantly jumping all over the track. By keeping track of the amount of error with respect to time, the system is able to be adjusted more appropriately with the behavior of the vehicle.

H. Steady State Error

Steady State Error is defined as the difference between the desired and the actual output as time goes to infinity [12]. This refers to the time in a transient response after the response rises and settles to its final value. This is the steady state value. The error comes in when this final value differs from the desired value decided from the design of the system. Below, in Figure 4, we can see an example of a unit step input and its resulting output.

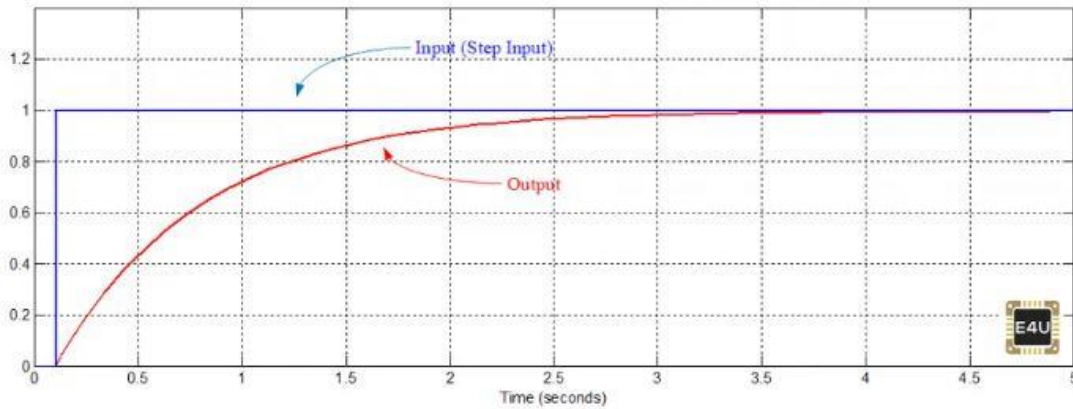


Figure 4: Example of Unit Step Input with Steady State Error of 0 [12]

This system produces an output that has a relatively long rise time, but the final value is identical to the input value. This results in a steady state error of zero. With further testing, it will be found that step inputs are the lowest in steady state error, while parabolic inputs are the highest in steady state error. Ramp functions are usually somewhere in between the two.

In our system we give a certain input to the steering system, and we have an output that we expect based on the transfer function for our system. The thing that we have to try and manage is the range of error of this output. In our implementation, we have to write code to manage error. This code can be seen in the appendix. We calculate error based on the difference of the two signals that the microcontroller reads from the sensor circuits. We then take this error difference and compare it to a positive or negative value that we calculated beforehand, with each value

representing both extremes of the steering range. Based on how the error compares to one of these values, a signal is sent to the servo that tells it to turn a certain angle to correct for this amount of error. This method, however, can still leave room for steady state error because the resulting servo angle from any given signal may not have the same final value that we assume it will have. Because of this, the microcontroller has to constantly adjust based on the position of the car on the track.

I. Derivative Kick

In the PD Control System, the value of error changes suddenly when the set point is adjusted [4]. Since instantaneous jump is similar to a vertical line, the slope of this is a very large number. This large number causes the controller to receive false values, and it will saturate for one cycle at one of the lower/upper bounds, and possibly disturb the system [4].

One method to handling this problem is the set point elimination [5]. In this method, the derivative is not calculated directly from the error, but is calculated from the negative of the process variable [5]. The two calculations will be equivalent assuming the setpoint is constant [5]. This causes the derivative that is based off of the negative process variable to be accurate and also lacking in the sudden jump in value [5].

III Experimental or Procedure

A. Steering Control Design

The first step in designing the steering control is to analyze the MATLAB code, given in section A of the Appendix, in order to understand it fully. The given code is incomplete, as there are a few values that need to be input into the code from previous labs, as well as estimated in the design process. The state equations, along with the transfer function estimation from the previous lab should be put where the comments suggest. Based on formula sheets, the gear ratio and ADC conversion value can be estimated. The ratio of distance to voltage variable (called ‘SenSlope’ in the code) can be taken from the Lab 1 results, and the transfer function estimation can be taken from the Lab 2 results.

The next values needed in the code are the values of crossover frequency and phase margin. These values are chosen in order to make a stable system, using techniques mentioned in the theory section of this report. The values of K_P and K_D are calculated by the equations in Section C of the Appendix and input into the code where needed. These K-values should be written down for later use in the lab. When all the necessary information is filled in, the code can be run, and the values of crossover frequency and phase margin can be adjusted as needed in order to produce a better step response or more stable system. The code produces the gain and phase Bode plots of the open-loop system, a Nyquist plot of the open-loop system, and a step response of the closed-loop.

B. Steering Control Simulation

After the MATLAB code is done and completely filled out, the Simulink block diagram can be created. It can only be done with a MATLAB code connected to it, in order to use the values from the MATLAB code. First starting out with placing a step input at the beginning of the diagram,

with the arguments of the step time being equal to 1, the initial value being 0, the final value being 1, and the sample time being 0. Next, connecting it to a gain block with the gain being the ADC value from the MATLAB code. Using a summation block to add it with feedback that will later be explained, but that summation is the input to a discrete PD controller. The parameters of the PD controller is that it uses a sample time of 0.001, the proportional and derivative constants are K_p and K_d , respectively, and for the saturation, the upper limit is positive $\pi/2$ and the lower limit is negative $\pi/2$. After that, the signal is connected to the servo transfer function. After the transfer function, a summation of the transfer function and a disturbance signal with a constant value of $\text{deg2rad}(2)$ is connected to the gear ratio gain. Connected to that is the dynamic state-space equations.

Now working on the feedback loop, the loop starts from the dynamic state-space equations and is connected to a gain of the sensor slope. The sensor slope gain is connected to a zero-order hold with a sample time of -1. That is then finally connected to the ADC gain and connected to the first summation block with a negative sign.

Finally, adding two scopes to the block diagram to produce graphs. The first scope is connected to three points. The first point is the connection between the first ADC gain and the first summation block. The second point is the connection between the first summation block and the PD controller block. The third point is the end of the feedback loop. The second scope is the output of the entire system, which is the end of the dynamic state-space block. Figure XX shows the completed block diagram.

C. Control Implementation

Once the values of K_D and K_P are estimated, they can be input into the code in section B of the Appendix. This code will run through the microcontroller and respond in accordance to the distance between the track and the sensors. After the code is finalized, the sensors should be attached to the car so that the inductors are perpendicular to the track, as well as the programmed microcontroller. The car must then be pushed around the track to show the control system. The values of K_P and K_D can be further adjusted to make the steering system more accurate.

IV Discussion / Results

A. MATLAB Output

The output from the MATLAB code included a Bode Plot, Nyquist plot, and step response. In order to do so, certain values had to be chosen in order to make the system stable. The phase margin was chosen to be 0.52 rad/s, and the phase crossover frequency was chosen to be 68 degrees. This led to the K_D value to be 10.0947, and the K_P value to be 11.8742.

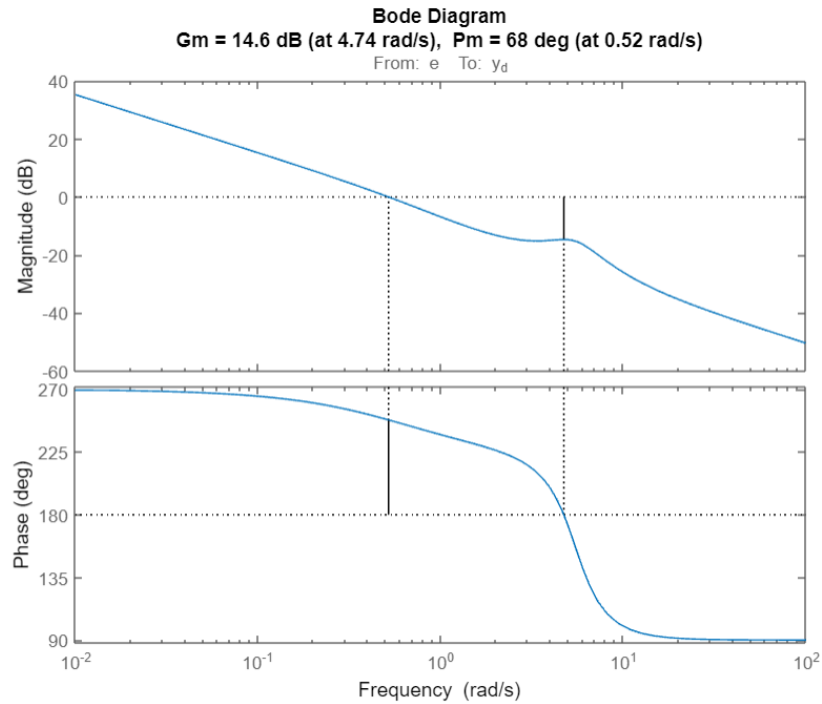


Figure 5: Bode Plot of the Open-Loop System

The figure above shows the first output of the MATLAB code, which is the magnitude (top) and phase (bottom) Bode Plots for the open-loop system. The gain margin, gain crossover frequency, phase margin, and phase crossover frequency can all be found from the graph. Along with being listed on the top of the graph, they can be ascertained from the plots themselves.

The length of the solid vertical black line on the magnitude plot represents the gain margin, and the x-value that the line is at represents the gain crossover frequency. The phase margin is the length of the similar vertical line on the phase plot, and the frequency that it lies at is the phase crossover frequency. These values are both positive, which shows that the system is stable.

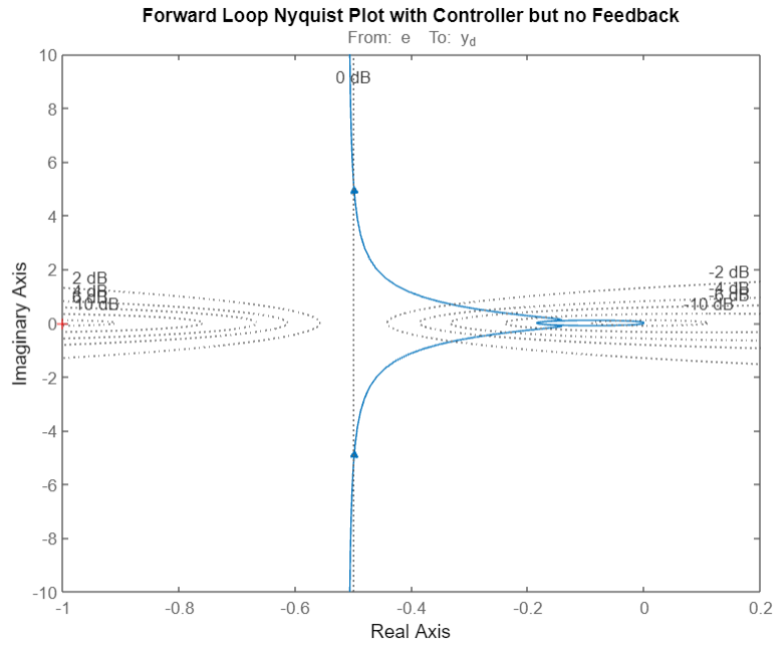


Figure 6: Nyquist Plot of the Open-Loop System

The figure above shows the Nyquist plot of the open-loop steering system. The formula from Equation 2, as well as the graph above can be used to find that the system is stable.

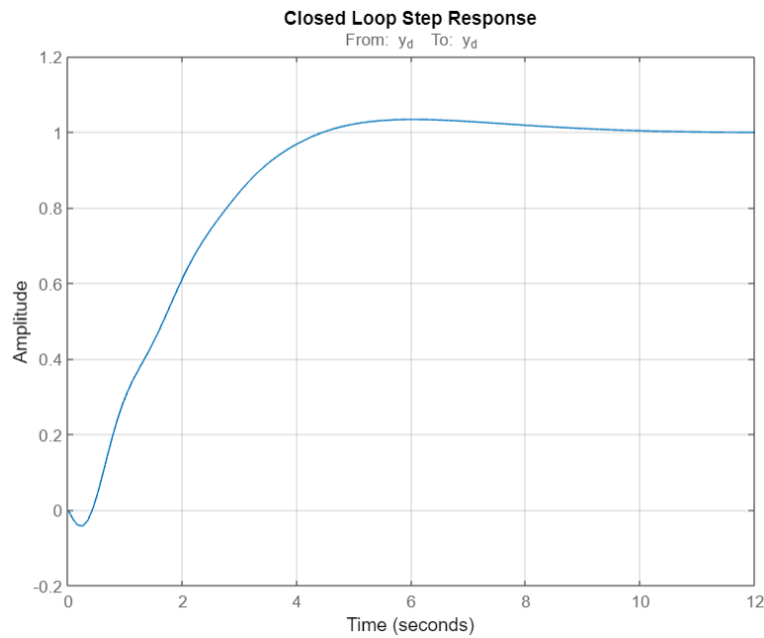


Figure 7: Step Response of Closed-Loop System

The figure above shows the step response of the closed-loop system, which is an output of the MATLAB code. The tradeoff between robustness and speed can be seen in this response, as changing the phase margin and phase crossover frequency will alter the rise time and overshoot.

B. Simulink Output

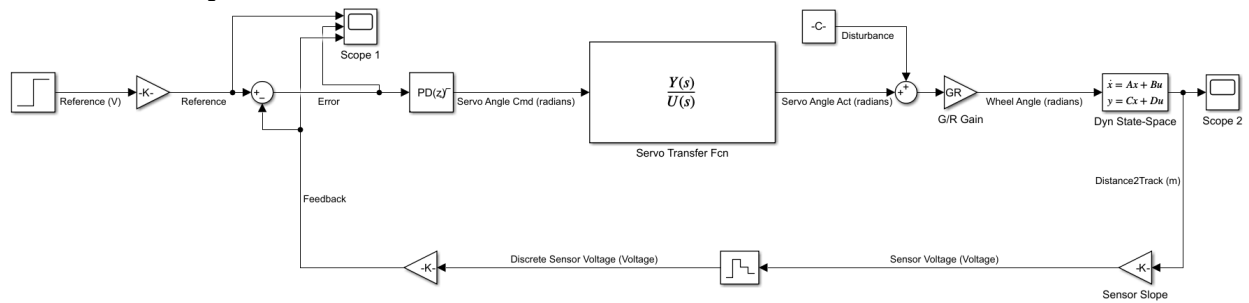


Figure 8: Simulink Block Diagram

Figure 8 above shows the block diagram created in Simulink. This schematic is what was used to design the control system, the outputs of which are shown in the below figures.

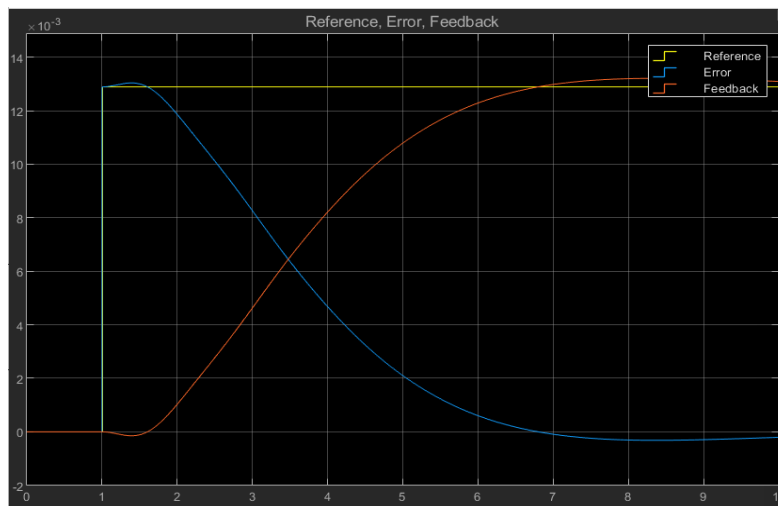


Figure 9: Simulink Output Without Disturbance

After the schematic in Figure 8 was built (excluding the constant addition of 2 degrees), the graphs above in Figure 9 were produced. The yellow signal shows the input step response, the red signal shows the output response, and the blue shows the error signal.

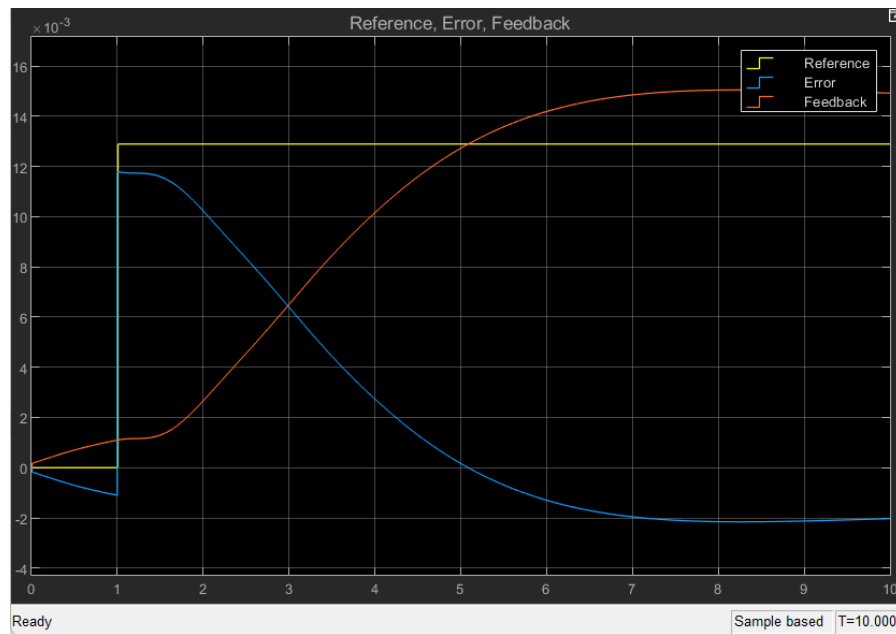


Figure 10: Simulink Output with Disturbance

A disturbance of 2 degrees was added to the Simulink block diagram, which can be seen in Figure 8. The output of this disturbance, which mimics what would happen if the system was not centered initially, is seen above in Figure 10. The yellow signal represents the input step response, the red signal shows the output, and the blue shows the error signal. When comparing Figures 9 and 10, a large difference can be seen. This can lead to the need for an integral aspect to be added to the control system, along with the already present derivative and proportional.

V Conclusions

Conclusion

All of the objectives of this lab were completed. We started with the MATLAB code, and produced a Bode plot, a Nyquist plot, and a step response. All three of these results showed that our system was stable. After that, we moved on to the Simulink part. We successfully constructed a block diagram to represent our system. This would produce a graph showing a step response, an output response of the system, and output error. We then implemented a small disturbance to see how it would affect the system. It ended up having a large impact on the output of the system. This means that later on, we will need to add more control to the system to account for potential disturbances. This lab confirmed to us, in a couple of different ways, the stability of our system, which is very important to know going forward. We also began to explore ways that our system can be lacking in robustness when it comes to disturbance. This data will help us in the continuing design of our system.

VI Appendix

A. MATLAB Code

```

%% Close open windows and clear command window
close all;
clear all;
clc;

%% Dynamic Coefficients
m = 0.538;%insert value here      %Kilograms mass of the car
L_z = 0.004676;%insert value here %Nm Moment about the Z axis
L_f = 0.2159;%insert value here   %m length between the COM and the
front axel
L_r = 0.0254;%insert value here   %m length between the COM and the rear
axel
C_alphaf = 20E3;%insert value here %N/rad Cornering Stiffness of the Front
tire
C_alphar = 20E3;%insert value here %N/rad Cornering Stiffness of the Rear
tire
V_x = 1;%insert value here       %m/s constant speed of the car

ds = 0.1614;%insert value here    %m distance from COM to the sensor

%% Transfer function of the Dynamic Model
A = [0, 1, 0, 0;...
      0, -(2*C_alphaf+2*C_alphar)/(m*V_x), 0, -V_x-(2*L_f*C_alphaf-
2*L_r*C_alphar)/(m*V_x);...
      0, 0, 0, 1;...
      0, -(2*L_f*C_alphaf-2*L_r*C_alphar)/(L_z*V_x), 0, -
(2*((L_f)^2)*C_alphaf+2*((L_r)^2)*C_alphar)/(L_z*V_x)];%insert value here
B = [0; (2*C_alphaf)/m; 0; (2*L_f*C_alphaf)/L_z];%insert value here
C = [0,1,ds,0];%insert value here
D = [0];%insert value here

G_dynamicsSS = ss(A,B,C,D,'StateName', {'e_1' 'e_2' 'e_3' 'e_4'},...
'InputName','delta','OutputName','y');
G_dynamics = tf(G_dynamicsSS)

%% Servo Coefficients
%%Second Order Lag Approximation
%{
omega_0 = ;%insert value here %resonant frequency
zeta = ;%insert value here    %damping ratio
G = ;%insert value here       %steady-state gain

num = G*omega_0^2;
denom = [1, 2*zeta*omega_0, omega_0^2];
%}

%%Second Order Delay Approximation
% %{
%T_delay = .3618;%insert value here %
%N_order = 2;%insert value here %
%[num,denom] = pade(T_delay, N_order);
%}

%%Matlab Transfer Function Approximation

num = [-6.239, 9.676, 64.25]; %insert value here
denom = [1, 6.048, 40.46, 81.07, 39.91]; %insert value here

%% Transfer function of the estimated Servo Motor
G_smotor = tf(num,denom, 'InputName','u', 'OutputName','delta');

%% Gains of the forward loop
GR = 0.53; %insert value here      %Gear Ratio from servo angle
to turn angle
ADC = 3.3/256; %insert value here   %ADC conversion voltage to
ADC#
SensSlope = 6.67; %insert value here %Ratio from distance to
voltage

%%Open-loop transfer function including forward gain
G_Feedback = tf(ADC*SensSlope,'InputName','y', 'OutputName','y_d');
%Feedback path gain
G_plant = G_dynamics*GR*G_smotor;
G_forward = G_Feedback*G_plant; %Forward path gain
%% Bode plot of the open-loop transfer function including forward gain
figure;
bode(G_forward); %insert transfer function
title('Open Loop Bode Plot');
grid on;

%% Declaring wc and r453rwm
wc = 0.52; %insert crossover frequency (rad/s)
wm = deg2rad(68); %insert phase margin

%% Frequency response
F = freqresp(G_forward, wc); %insert [transfer function, wc]
magGforward = abs(F); %magnitude for PI controller
angleGforward = angle(F); %phase for PI controller

%% Calculating parameters for the PI controller
Kp = (1/(magGforward))*cos(pi+wm-angleGforward) %insert equation
Kd = (1/(wc*magGforward))*sin(pi+wm-angleGforward) %insert equation

%% Including the PD controller in the control loop
num1 = [Kd, Kp]; %insert Kp and Kd in transfer function form
denom1 = [1]; %insert Kp and Kd in transfer function form

G_c = tf(num1,denom1,'InputName','e', 'OutputName','u');
G_open = G_forward*G_c; %open-loop + controller transfer
function

%% Margin plot with the PD controller but not the feedback loop
figure;
margin(G_open);
title('Forward Loop Bode Plot with Controller but no Feedback');
%grid on;

%% Nyquist plot with the PD controller but not the feedback loop
figure;
nyquist(G_open); %insert transfer function
title('Forward Loop Nyquist Plot with Controller but no Feedback');
grid on;

%% Pole Zero plot with the PD controller but not the feedback loop
figure;
pzplot(G_open); %insert transfer function
title('Forward Loop Pole Zero Plot with Controller but no Feedback');
grid on;

%% The complete transfer function
%Closed Loop Transfer Function from equation

Gclose = G_open/(1+(G_open)); %closed loop system transfer function

%Closed Loop Transfer Function from functions in control toolbox
% %{
sum = sumblk('e = r_d - y_d'); %Summation block from block diagram
ADC_in = tf(ADC, 1, 'InputName','r', 'OutputName','r_d'); %reference ADC
input
G_closed = connect(ADC_in, G_c, G_plant, G_Feedback, sum, 'r','y');
%connect all blocks
%}

%% Pole Zero plot of the closed loop system
figure;
pzplot(G_closed); %insert transfer function
title('Forward Loop Pole Zero Plot with Controller but no Feedback');
grid on;

%% Step response of the closed loop
figure;
step(Gclose); % plot the step response of Gclose
title('Closed Loop Step Response');
grid on;

```


B. Mbed Code

```

#include "mbed.h"
#include <stdio>
#include <iostream>
#define TI 0.001 //1kHz sample time
#define KP 5.0f //Proportional Gain
#define KD 0.001f //Integral Gain
PwmOut PWM1(PTA1);
AnalogIn senseLeft(PTB1);
AnalogIn senseRight(PTB2);
float errorChange;
float error0;
float error1;
float controllerOutput;
float dutyCycle;

int main(){
    error0=0;
    while(true){
        error1 = senseLeft.read()-senseRight.read();
        printf("\nerror1: \n");

        printf("%f", error1);

        errorChange = ((error1-error0)/TI);
        controllerOutput= KP*error1 + KD*errorChange;
        if(controllerOutput>1.57){
            controllerOutput=1.57; }

        else if(controllerOutput < -1.57){
            controllerOutput=-1.57; }
        printf("\nController Output: \n");
        printf("%f", controllerOutput);

        dutyCycle=controllerOutput*0.0159+0.075;
        PWM1.write(dutyCycle);
        printf("\nduty cycle: \n");
        printf("%f", dutyCycle);
        PWM1.write(dutyCycle);
        error0=error1;
        printf("\nerror0: \n");
        printf("%f", error0);
        wait_us(1000000*TI);}
    }

```

C. PD Control Design Derivation

$$G(s) = G_p(s) * H(s) \quad \text{Line 1}$$

$$G_{FP}(s) = G_C(s) * G(s) \quad \text{Line 2}$$

$$|G(j\omega_c)|e^{j\angle G(j\omega_c)} \quad \text{Line 3}$$

$$G_{FP}(j\omega_c) = e^{j(\pi+\varphi_m)} \quad \text{Line 4}$$

$$G_C(j\omega_c) * |G(j\omega_c)|e^{j\angle G(j\omega_c)} = e^{j(\pi+\varphi_m)} \quad \text{Line 5}$$

$$G_C(j\omega_c) = \frac{e^{j(\pi+\varphi_m)}}{|G(j\omega_c)|e^{j\angle G(j\omega_c)}} = \frac{e^{j(\pi+\varphi_m-\angle G(j\omega_c))}}{|G(j\omega_c)|} \quad \text{Line 6}$$

$$e^x = \cos x + j\sin x \quad \text{Line 7}$$

$$K_p + sK_d = K_p + j\omega_c K_d = \frac{\cos(\pi+\varphi_m-\angle G(j\omega_c)) + j\sin(\pi+\varphi_m-\angle G(j\omega_c))}{|G(j\omega_c)|} \quad \text{Line 8}$$

$$K_p = \frac{\cos(\pi+\varphi_m-\angle G(j\omega_c))}{|G(j\omega_c)|} \quad \text{Line 9}$$

$$j\omega_c K_d = \frac{j\sin(\pi+\varphi_m-\angle G(j\omega_c))}{|G(j\omega_c)|} \quad \text{Line 10}$$

$$K_d = \frac{\sin(\pi + \varphi_m - \angle G(j\omega_c))}{\omega_c |G(j\omega_c)|}$$

Line 11

VII References

- [1 A. Monti and A. Smith, "ELCT 302 Real Time Systems Desgin Laboratory Syllabus," Fall 2008. [Online]. Available: <http://www.ee.sc.edu/classes/Fall08/elct401/>.
- [2 Electronics Coach, "Proportional Derivative (PD) Controller," [Online]. Available: <https://electronicscoach.com/proportional-derivative-controller.html>. [Accessed 1 March 2023].
- [3 V. Sandeep, "When and why to use P, PI, PD, and PID Controller?," 6 October 2021. [Online]. Available: <https://medium.com/@svm161265/when-and-why-to-use-p-pi-pd-and-pid-controller-73729a708bb5>. [Accessed 1 March 2023].
- [4 Dynamics and Control, "Proportional Integral Derivative (PID)," 31 December 2022. [Online]. Available: <https://apmonitor.com/pdc/index.php/Main/ProportionalIntegralDerivative#:~:text=Derivative%20kick%20occurs%20because%20the,an%20upper%20or%20lower%20bound..> [Accessed 1 March 2023].
- [5 V. Vandoren, "The Three Faces of PID," 1 March 2007. [Online]. Available: <https://www.controleng.com/articles/the-three-faces-of-pid/>. [Accessed 1 March 2023].
- [6 Electrical4U, "Bode Plot, Gain Margin and Phase Margin," 24 January 2021. [Online]. Available: <https://www.electrical4u.com/bode-plot-gain-margin-phase-margin/>. [Accessed 1 March 2023].
- [7 N. S. Nise, Control Systems Engineering, California : John Wiley & Sons, 2015.
- [8 Tutorialspoint, "Control Systems - Nyquist Plots," [Online]. Available: https://www.tutorialspoint.com/control_systems/control_systems_nyquist_plots.htm#:~:text=Nyquist%20stability%20criterion%20states%20the,of%20the%20's%20plane.. [Accessed 1 March 2023].
- [9 Wolfram MathWorld, "Taylor Series," Wolfram, 24 February 2023. [Online]. Available: <https://mathworld.wolfram.com/TaylorSeries.html>. [Accessed 1 March 2023].
- [1 Electrical4U, "Transient And Steady State Response," 25 October 2020. [Online]. Available: <https://www.electrical4u.com/transient-state-and-steady-state-response-of-control-system/>.

- [1 D. Collins, "Motion Control Tips," WTW Media LLC, 16 June 2016. [Online]. Available:
1] <https://www.motioncontroltips.com/faq-crossover-frequencies-used-servo-motor-tuning/>.
[Accessed 13 March 2023].
- [1 D. V. Jain, "Steady State Error Analysis," Electrical4U, 22 October 2020. [Online]. Available:
2] <https://www.electrical4u.com/steady-state-error-analysis/>. [Accessed 14 March 2022].
- [1 "Electronics Coach," [Online]. Available: <https://electronicscoach.com/proportional-derivative-controller.html#:~:text=A%20type%20of%20controller%20in,derivative%20controller%20or%20PI%20controllers..> [Accessed 14 March 2023].
- [1 WikiBooks, "Control Systems/Discrete-Time Stability," WikiBooks, 14 June 2017. [Online].
4] Available: https://en.wikibooks.org/wiki/Control_Systems/Discrete-Time_Stability.
[Accessed 14 March 2022].